INFRASTRUKTURA — V1.0

ClientOps

Platforma CRM nowej generacji

Wymagania Infrastrukturalne

Analiza wymagań sprzętowych i architektonicznych dla 10, 100 i 5 000 klientów. System ciągłego monitoringu SLA, tygodniowej analizy SEO z wielu źródeł (GSC, GA4, PageSpeed, AI) oraz miesięcznych audytów z generowaniem PDF.

WERSJA	DATA	SCENARIUSZE	TYP
1.0.0	Marzec 2026	3	DevOps / SRE

Spis treści

Wymagania Infrastrukturalne — ClientOps CRM

1 Charakterystyka Systemu i Model Obciążenia

2 Scenariusz: 10 Klientów — Single Server

- Specyfikacja serwera
- Konfiguracja Horizon
- Szacunkowy koszt

3 Scenariusz: 100 Klientów — Mały Klaster

- Architektura 2–3 serwerów
- Specyfikacja komponentów
- Konfiguracja Horizon

4 Scenariusz: 5 000 Klientów — Klaster Produkcyjny

- Architektura HA z load balancerem
- Specyfikacja wszystkich węzłów
- Konfiguracja Horizon i kolejek
- TimescaleDB — partycjonowanie i retencja

5 Krytyczne Indeksy Bazy Danych

6 Wąskie Gardła i Optymalizacje

- PDF Generation (Browsershot)
- Monitoring SLA — skalowanie zapytań
- Google APIs — rate limiting
- AI Processing — koszt i cache

7 Zestawienie Kosztów i Rekomendacje

1

PODSTAWY

Charakterystyka Systemu i Model Obciążenia

Stałe procesy działające w tle

ClientOps nie jest systemem pasywnym — kilka procesów działa **nieprzerwanie**, niezależnie od aktywności użytkowników:

PROCES	CZĘSTOTLIWOŚĆ	KOMPONENT	OBCIĄŻENIE
`CheckSlaBreachesJob`	`co 5 minut`	Laravel Scheduler → Queue	Zapytanie DB — skanuje aktywne tickety i projekty
Analiza SEO — pipeline	`1×/tydz. per projekt`	Horizon (5 kolejek)	I/O sieciowy, Google APIs, AI API, DB write
Miesięczny audyt + PDF	`1×/miesiąc per klient`	Horizon + Browsershot	Headless Chromium — ~1 GB RAM / instancja
WebSocket (Reverb)	`ciągłe połączenia`	Laravel Reverb + Redis	Długotrwałe połączenia TCP, pub/sub Redis
Indeksowanie Meilisearch	`po każdej zmianie`	Scout Observer → Queue	Lekkie — małe incremental updates

Pipeline SEO — 7 jobów per analiza

Każda analiza SEO jednego projektu generuje sekwencję jobów w różnych kolejkach:

```
AuditRequestedEvent └→ CrawlWebsiteJob queue: crawlers | Crawling stron, zbieranie meta/h1/broken
links └→ FetchGSCDataJob queue: google-apis | Google Search Console API – pozycje, kliknięcia └→
FetchGA4DataJob queue: google-apis | Google Analytics 4 API – ruch organiczny (równolegle) └→
AnalyzeKeywordsJob queue: ai-processing | Analiza słów kluczowych + PageSpeed Insights └→
AnalyzeSemanticsJob queue: ai-processing | Analiza semantyczna treści przez Claude/OpenAI
(równolegle) └→ GenerateAuditReportJob queue: pdf | Browsershot PDF – headless Chromium └→
NotifyAuditCompleteJob queue: default | Mail + in-app notification
```

Model obciążenia — założenia bazowe

PARAMETR	WARTOŚĆ	UZASADNIENIE
Projekty per klient (śr.)	3	Agencja obsługuje kilka stron/projektów
Analizy SEO/projekt/miesiąc	4 (tygodniowo)	Cotygodniowy snapshot pozycji i wskaźników
Joby per analiza SEO	7	Pipeline: crawl + 2×Google + 2×AI + PDF + notify
Joby per miesięczny audyt	3	GenerateReport + BrowsershotPDF + Notify
SLA scany/dobę	288	Co 5 minut × 24h = 288 uruchomień schedulera
Aktywni użytkownicy per klient	2	Admin + manager (lub client-admin + client-user)
Czas generowania PDF	30–90s	Headless Chromium, zależy od złożoności raportu
RAM per instancja Browsershot	~1 GB	Chromium to ciężki proces — każdy worker = 1 Chromium

ℹ **Google APIs — rate limity**

Google Search Console API: 200 zapytań/100 sekund per projekt (OAuth2 user token). Przy dużej liczbie projektów wymagane jest rozłożenie jobów w czasie oraz retry logic z exponential backoff w kolejce `google-apis`.

2

SCENARIUSZ STARTOWY

10 Klientów — Single Server

STARTER / DEV / PILOT

10 klientów · 30 projektów · 5–20 aktywnych userów

Idealne na start, pilotaż, środowisko developerskie lub bardzo małą agencję. Wszystkie komponenty na jednym serwerze.

Profil obciążenia

30	~28	~1	288	5–20
projektów	joby SEO/dzień	audyt PDF/dzień	SLA scany/dzień	concurrent users

~840
joby SEO/miesiąc

Przy 10 klientach obciążenie jest minimalne. SLA job skanuje kilkadziesiąt rekordów, kolejki SEO pracują głównie w nocy (zaplanowane), a jeden Chromium wystarcza do generowania ~1 PDF dziennie.

Architektura: pojedynczy serwer VPS

```
┌──────────────────────────────────────────────────────────────────────────────┐ │ VPS — Jeden
Serwer │ │ │ │ Nginx reverse proxy, SSL termination, gzip │ │ PHP-FPM Laravel 12, PHP 8.3, 8
workers │ │ PostgreSQL 16 + TimescaleDB extension — główna baza danych │ │ Redis 7 cache + queue
driver + sesje + pub/sub Reverb │ │ Horizon 5 kolejek, 8 workerów łącznie │ │ Reverb WebSocket
server (kilka jednoczesnych połączeń) │ │ Meilisearch full-text search, ~100MB RAM │ │ Chromium
Browsershot — 1 instancja, uruchamiana na żądanie │ │ │
└──────────────────────────────────────────────────────────────────────────────┘
```

Specyfikacja serwera

Serwer Aplikacji + Bazy

vCPU	4 cores
RAM	8 GB
Dysk	100 GB NVMe SSD
Transfer	2 TB / miesiąc
OS	Ubuntu 22.04 LTS
Przykład (Hetzner)	CX21 / CX31

Podział pamięci RAM (8 GB)

PostgreSQL	3 GB (shared_buffers)
PHP-FPM workers	~1.5 GB
Redis	512 MB (maxmemory)
Horizon workers	~1 GB
Meilisearch	~256 MB
Chromium (peak)	~1 GB (chwilowo)

Konfiguracja Horizon

default

2

workery

CRUD, notifikacje

crawlers

1

worker

I/O intensive

google-apis

1

worker

rate limited

ai-processing

1

worker

OpenAI / Claude

pdf

1

worker

~1 GB RAM

Szacunkowy koszt miesięczny (Hetzner)

30–60 EUR / miesiąc

CX31 (4vCPU, 8GB) ≈ €12/mies. + backup €2 + MinIO/S3 storage + domeny

✓ **Kiedy ten scenariusz jest wystarczający**

Do ~15 klientów single-server działa bez problemów. Uwaga: Chromium generujący PDF i PHP-FPM rywalizują o CPU — zaplanuj generowanie PDF poza godzinami szczytu (np. 2:00–6:00 rano przez schedulera Laravel).

3

SCENARIUSZ WZROSTU

100 Klientów — Mały Klaster

🚀

GROWTH / MAŁA AGENCJA SAAS

100 klientów · 300 projektów · 50–200 aktywnych userów

Separacja warstw — baza danych na dedykowanym serwerze, osobna pula workerów Horizon. System potrzebuje 2–3 maszyn.

Profil obciążenia

300	~280	~10	~12	50–200	~8 400
projektów	joby SEO/dzień	audytów PDF/dzień	joby SEO/godz.	concurrent users	joby SEO/miesiąc

Architektura: 2–3 serwery

```
Internet │ ┌──────────────┐ │ App Server │ │ Nginx │ │ PHP-FPM │ │ Laravel Horizon│ ← workery
SEO, PDF, API │ Reverb WS │ │ Meilisearch │ └──────┬──────┘ │ ┌──────────┴──────────┐ │ │
┌──────────────┐ ┌──────────────┐ │ DB Server │ │ Redis Server │ │ PostgreSQL 16 │ │ Redis 7 │
│ TimescaleDB │ │ (sessions, │ │ 500 GB NVMe │ │ queue, cache, │ │ │ │ pub/sub) │
└──────────────┘ └──────────────┘
```

Specyfikacja komponentów

🖥️ App Server

vCPU	8 cores
RAM	16 GB
Dysk	100 GB NVMe SSD
Przykład	Hetzner CCX23
Koszt	~€55/mies.

🗄️ DB Server

vCPU	8 cores
RAM	32 GB
Dysk	500 GB NVMe SSD
Przykład	Hetzner CCX33
Koszt	~€95/mies.

⚡ Redis Server

vCPU	4 cores
RAM	8 GB
Dysk	60 GB SSD
Przykład	Hetzner CX31
Koszt	~€15/mies.

Konfiguracja Horizon

default

4

workery

CRUD, notifikacje, Scout

crawlers

2

workery

I/O intensive | timeout: 300s

google-apis

2

workery

rate limited | throttle middleware

ai-processing

2

workery

OpenAI / Claude API

pdf

2

workery

2× Chromium ~2GB RAM

⚠ **PDF Queue — uwaga na pamięć**

2 workery PDF × ~1 GB Chromium = ~2 GB RAM tylko na generowanie PDF. Na App Serverze z 16 GB to akceptowalne, ale nie zwiększaj workerów PDF powyżej 4 bez upgrade'u RAM do 32 GB.

Szacunkowy koszt miesięczny (Hetzner)

175–350 EUR / miesiąc

App €55 + DB €95 + Redis €15 + backup + storage MinIO/S3 + domeny SSL

4

SCENARIUSZ PRODUKCYJNY

5 000 Klientów — Klaster HA

🏭

SCALE / SAAS PRODUCTION

5 000 klientów · 15 000 projektów · 1 000–5 000 użytkowników (peak)

Pełny klaster HA z load balancerem, wieloma węzłami app, replikami DB, Redis cluster i dedykowanymi workerami kolejek.

Profil obciążenia

15 000 projektów	**~14 000** joby SEO/dzień	**~500** audytów PDF/dzień	**~583** joby SEO/godz.	**1–5k** concurrent users
~500–2k WS connections				

🔴 **500 PDF/dzień = ~21 audytów/godzinę przez Chromium**

Przy 4 workerach PDF każdy zajęty przez 30–90 sekund — przepustowość to ~160–480 PDF/godz. Potrzeba minimum 4–6 workerów PDF na dedykowanym serwerze z 32 GB RAM, aby nie tworzyć zaległości w kolejce.

Architektura klastra

```
Internet / CDN │ ┌──────────────────────┐ │ Load Balancer × 2 │ (HA Pair — HAProxy lub Nginx) │
(Active / Passive) │ Failover automatyczny └──────────┬───────────┘ │ ┌──────────┴──────────┐
│ │ │ ┌──────────┐ ┌──────────┐ ┌──────────┐ │App Node 1│ │App Node 2│ │App Node 3│ ← PHP-FPM +
Nginx │ Nginx │ │ Nginx │ │ Nginx │ Session: Redis │ PHP-FPM │ │ PHP-FPM │ │ PHP-FPM │ Cache: Redis
│ Reverb │ │ Reverb │ │ │ Stateless └────┬─────┘ └────┬─────┘ └────┬─────┘
└──────────┴──────────┘ │ (czyta z repliki, pisze do primary)
┌──────────┴──────────┐ │ │ ┌──────────────┐ ┌──────────────────┐ │ DB Primary
│ │ DB Replicas × 2 │ │ PostgreSQL 16 │ ──► │ (read-only) │ │ TimescaleDB │ │ SELECT queries, │ │
16 vCPU/64 GB │ │ raportowanie, exports │ │ 2 TB NVMe SSD │ │ 16 vCPU/64 GB each │
└──────────────┘ └──────────────────┘ ┌──────────────────────────────────┐ │
Redis Cluster (6 węzłów: 3 primary + 3 replika)│ │ Node: 4 vCPU / 16 GB RAM / 50 GB SSD each │ │
Cluster mode — sharding kluczy │ └──────────────────────────────────┘
┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ Horizon Server │ │ Crawler
Workers │ │ Reverb / WS │ │ 16 vCPU / 32 GB │ │ × 2–3 nodes │ │ × 2 nodes │ │ dispatcher + │ │ 8
vCPU / 16 GB │ │ 4 vCPU / 8 GB │ │ worker processes │ │ sieciowe I/O │ │ Redis pub/sub │
└──────────────┘ └──────────────┘ └──────────────┘ ┌──────────────┐
┌──────────────┐ │ Meilisearch × 2 │ │ MinIO Cluster │ │ 8 vCPU / 32 GB │ │ × 4 nodes │ │ 200
GB SSD │ │ 4 vCPU / 8 GB │ │ primary + replica│ │ 1 TB HDD per node │ └──────────────┘
└──────────────┘
```

4

SCENARIUSZ 5 000 KLIENTÓW — CIĄG DALSZY

Specyfikacja węzłów i konfiguracja Horizon

Specyfikacja wszystkich węzłów

⚖️ Load Balancer × 2

vCPU	2 cores
RAM	4 GB
Dysk	40 GB SSD
Koszt	2 × €8 = €16

🖥️ App Nodes × 3

vCPU	8 cores
RAM	16 GB
Dysk	100 GB NVMe SSD
Koszt	3 × €55 = €165

🗄️ DB Primary

vCPU	16 cores
RAM	64 GB
Dysk	2 TB NVMe SSD
Koszt	~€320/mies.

📖 DB Replicas × 2

vCPU	16 cores
RAM	64 GB
Dysk	2 TB NVMe SSD
Koszt	2 × €320 = €640

⚡ Redis Cluster × 6

vCPU	4 cores / węzeł
RAM	16 GB / węzeł
Dysk	50 GB SSD / węzeł
Koszt	6 × €35 = €210

⚙️ Horizon Server

vCPU	16 cores
RAM	32 GB
Dysk	100 GB NVMe SSD
Koszt	~€160/mies.

🕷️ Crawler Workers × 3

vCPU	8 cores
RAM	16 GB
Dysk	100 GB SSD
Koszt	3 × €55 = €165

📡 Reverb WS × 2

vCPU	4 cores
RAM	8 GB
Dysk	40 GB SSD
Koszt	2 × €15 = €30

🔍 Meilisearch × 2

vCPU	8 cores
RAM	32 GB
Dysk	200 GB NVMe SSD
Koszt	2 × €95 = €190

Konfiguracja Horizon (5 000 klientów)

default

10

workerów

CRUD, Scout, powiadomienia

crawlers

8

workerów

dedykowane węzły · timeout: 300s

google-apis

4

workerów

throttle Redis · exponential backoff

ai-processing

6

workerów

cache TTL 7 dni · timeout: 180s

pdf

6

workerów

6 GB RAM razem | timeout: 600s

TimescaleDB — konfiguracja dla 5 000 klientów

Tabela `seo_keyword_ranks` będzie rosnąć szybko: 15 000 projektów × średnio 50 słów kluczowych × 4 snapshoty/miesiąc = **3 000 000 rekordów/miesiąc**. TimescaleDB jest tu niezbędny.

```
-- Hypertable z partycjonowaniem miesięcznym
SELECT create_hypertable('seo_keyword_ranks', 'checked_at',
  chunk_time_interval => INTERVAL '1 month'
);

-- Polityka retencji: zachowaj 2 lata
SELECT add_retention_policy('seo_keyword_ranks', INTERVAL '2 years');

-- Continuous aggregate — tygodniowe trendy (dla wykresów echarts)
CREATE MATERIALIZED VIEW seo_rank_weekly
WITH (timescaledb.continuous) AS
  SELECT time_bucket('1 week', checked_at) AS week,
         keyword_id,
         project_id,
         AVG(position)    AS avg_position,
         MIN(position)    AS best_position,
         SUM(impressions) AS total_impressions
  FROM seo_keyword_ranks
  GROUP BY 1, 2, 3;

-- Polityka odświeżania agregatu
SELECT add_continuous_aggregate_policy('seo_rank_weekly',
  start_offset => INTERVAL '3 weeks',
  end_offset   => INTERVAL '1 hour',
  schedule_interval => INTERVAL '1 day'
);
```

Szacunkowy koszt miesięczny — self-hosted (Hetzner / OVH)

2 500 – 6 000 EUR / miesiąc

Alternatywa cloud (AWS/GCP z managed RDS, ElastiCache, S3): 5 000 – 15 000 EUR/mies.

5

OPTYMALIZACJA

Krytyczne Indeksy Bazy Danych

Bez odpowiednich indeksów system przy 5 000 klientach przestanie odpowiadać. Każde zapytanie w multi-tenant systemie zaczyna od filtrowania po `client_id` — to musi być zawsze zindeksowane.

Indeksy SLA — krytyczne dla CheckSlaBreachesJob

Job uruchamiany co 5 minut musi skanować tylko aktywne rekordy. **Partial index** dramatycznie zmniejsza rozmiar skanowanego zestawu.

```
-- Tylko aktywne tickety z przekroczonym terminem SLA
CREATE INDEX idx_tickets_sla_response
  ON tickets (sla_response_due_at)
  WHERE status NOT IN ('resolved', 'closed')
    AND sla_response_due_at IS NOT NULL;

CREATE INDEX idx_tickets_sla_resolution
  ON tickets (sla_resolution_due_at)
  WHERE status NOT IN ('resolved', 'closed')
    AND sla_resolution_due_at IS NOT NULL;

-- Nienotyfikowane naruszenia SLA
CREATE INDEX idx_sla_incidents_pending
  ON sla_incidents (breached_at)
  WHERE notified = false;
```

Indeksy Multi-tenancy — Global Scopes

Trait `BelongsToTenant` dodaje `WHERE client_id = ?` do każdego zapytania. Bez composite index każde zapytanie to full table scan.

```
-- Composite indeksy per tabela — najczęstsze kombinacje zapytań
CREATE INDEX idx_tickets_client_status      ON tickets (client_id, status);
CREATE INDEX idx_projects_client            ON projects (client_id);
CREATE INDEX idx_leads_client_status        ON leads (client_id, status);
CREATE INDEX idx_documents_documentable     ON documents (documentable_type, documentable_id);
CREATE INDEX idx_seo_analyses_project       ON seo_analyses (project_id, created_at DESC);
CREATE INDEX idx_audits_client_type         ON audits (client_id, type, created_at DESC);

-- Unikalność slugów w scope
CREATE UNIQUE INDEX idx_projects_slug_client    ON projects (client_id, slug);
CREATE UNIQUE INDEX idx_leads_slug_client       ON leads (client_id, slug);
CREATE UNIQUE INDEX idx_documents_slug          ON documents (documentable_type, documentable_id, s
```

Indeksy SEO — TimescaleDB

```
-- Dla zapytań o historię pozycji konkretnego słowa kluczowego
CREATE INDEX idx_seo_keyword_ranks_kw_time
  ON seo_keyword_ranks (keyword_id, checked_at DESC);

-- Dla dashboardu projektu — wszystkie słowa z ostatniego snapshotu
CREATE INDEX idx_seo_keyword_ranks_proj_time
  ON seo_keyword_ranks (project_id, checked_at DESC);

-- Dla alertów — gwałtowne spadki pozycji
CREATE INDEX idx_seo_issues_severity
  ON seo_issues (project_id, severity, created_at DESC)
  WHERE severity IN ('critical', 'warning');
```

Indeksy Wiadomości — Inbox Real-time

```
-- Nieprzeczytane wiadomości per uczestnik
CREATE INDEX idx_conv_participants_unread
  ON conversation_participants (participable_type, participable_id, last_read_at);

-- Wiadomości w konwersacji (stronicowanie)
CREATE INDEX idx_messages_conversation
  ON messages (conversation_id, created_at DESC);
```

6

ARCHITEKTURA

Wąskie Gardła i Strategie Optymalizacji

📄 PDF Generation — Browsershot / Chromium

Najcięższy pojedynczy job w systemie

⚠ **Problem:** Jeden Chromium zużywa ~1 GB RAM i 30–90 sekund CPU. Przy 500 PDF/dzień (5k klientów) kolejka `pdf` musi pracować non-stop przez 4–12 godzin.

✓ **Rozwiązanie:** Dedykowana kolejka `pdf` z limitem workerów = RAM / 1.2 GB. Planowanie generowania w oknach nocnych (00:00–06:00) przez Laravel Scheduler. Priorytetyzacja: audyty triggered ręcznie > automatyczne miesięczne.

Alternatatywa: wFPDF/DomPDF dla prostych raportów (szybsze), Browsershot tylko dla skomplikowanych layoutów z wykresami echarts.

⏰ SLA Monitoring — CheckSlaBreachesJob co 5 minut

Skalowanie zapytań przy dużej liczbie rekordów

⚠ **Problem:** Przy 5 000 klientach tabela `tickets` może mieć 100 000+ rekordów. Full table scan co 5 minut = pożeranie CPU i I/O bazy danych.

✓ **Rozwiązanie:** Partial index na aktywnych ticketach (patrz sekcja 5). Redis cache dla ostatnio sprawdzonych rekordów — skip jeśli sprawdzono < 5 min temu. Zamiast skanować całą tabelę: `WHERE sla_response_due_at < NOW() AND notified = false`.

Docelowo: zamiast pollingu co 5 min — event-driven (Carbon + Redis Sorted Set: ticket → score = timestamp breach). Job sprawdza tylko top N z ZRANGEBYSCORE.

🔑 Google APIs — Rate Limiting i Quota

GSC API: 200 req/100s per user token; GA4: 10 concurrent requests

⚠ **Problem:** Przy 15 000 projektach i 4 analizach/miesiąc = 60 000 wywołań GSC/miesiąc. Przy zbyt wielu równoległych workerach — API zwróci 429 Too Many Requests dla całej aplikacji.

✓ **Rozwiązanie:** Kolejka `google-apis` z throttle middleware opartym na Redis: max 20 jobów GSC/minutę per OAuth token. Exponential backoff przy 429. Rozłożenie jobów w czasie — nie planuj wszystkich analiz SEO na ten sam dzień tygodnia.

Każdy klient autoryzuje własny Google OAuth token — przy 100+ klientach limity dotyczą per token, nie globalnie. To znacznie zwiększa przepustowość.

AI Processing — Koszt i Latencja API

OpenAI / Claude — każda analiza semantyczna to koszt i czas

⚠ **Problem:** Analiza semantyczna jednej strony przez Claude/OpenAI: ~0.5–2 USD + 5–30 sekund latencji. Przy 15 000 projektach × 4/miesiąc = koszt może przekroczyć $30 000/miesiąc bez optymalizacji.

✔ **Rozwiązanie:** Cache wyników analizy semantycznej w Redis z TTL 7 dni — jeśli treść strony nie zmieniła się (hash treści), nie wysyłaj ponownie do AI. Batch processing: kilka stron w jednym promptcie tam gdzie możliwe. Limit analizowanych stron per projekt (np. top 10 stron wg ruchu z GA4).

Rozważyć modele: Claude Haiku lub GPT-4o-mini dla masowych analiz (tańsze 10×), Claude Opus/GPT-4 tylko dla finalnego raportu.

Crawler — Ethical Crawling i Timeouty

Crawlowanie cudzych stron musi szanować robots.txt i nie przeciążać serwerów klientów

⚠ **Problem:** CrawlWebsiteJob może trwać 10–300 sekund zależnie od rozmiaru strony. Jednoczesne crawlowanie 8+ stron = duże zużycie sieci i CPU. Brak respektu dla robots.txt lub zbyt agresywne crawlowanie może zablokować IP serwera.

✔ **Rozwiązanie:** Crawler workers na dedykowanych maszynach (osobne IP lub IP rotation). Respektowanie `robots.txt` i `Crawl-delay`. Limit: max 1 request/sekunda per domena. Timeout 300s z graceful cancellation. Cache crawl results na 24h — ponowne uruchomienie analizy nie crawluje od nowa.

Alternatywa dla własnego crawlera: zewnętrzne API (Screaming Frog API, Ahrefs API, SEMrush API) — drogie, ale eliminuje całkowicie problem crawlowania po stronie serwera.

7

PODSUMOWANIE

Zestawienie Kosztów i Rekomendacje

Porównanie scenariuszy

SCENARIUSZ	KLIENCI	PROJEKTY	JOBY SEO/DZIEŃ	PDF/DZIEŃ	SERWERY	KOSZT/MIES.
Starter	10	30	~28	~1	1	€30-60
Growth	100	300	~280	~10	3	€175-350
SCALE	5 000	15 000	~14 000	~500	20+	€2 500-6 000

Rekomendowana ścieżka wzrostu

KROK	KIEDY	DZIAŁANIE	TRIGGER
1. Single server	0-15 klientów	VPS 4 vCPU / 8 GB — wszystko na jednym serwerze	Start projektu
2. Separacja DB	~15-30 klientów	Przenieś PostgreSQL na dedykowany serwer	CPU DB > 60% lub RAM > 80%
3. Separacja Redis	~30-60 klientów	Redis na osobnym serwerze, dedykowane workery Horizon	Lag kolejki > 5 min
4. Drugi App Node	~80-120 klientów	Load balancer + 2 identyczne app nodes	Response time > 500ms lub deploy downtime nieakceptowalny
5. DB Replica	~200+ klientów	Read replica dla raportowania i SELECT-heavy queries	DB read IOPS > 80%
6. Redis Cluster	~500+ klientów	Redis Cluster mode (3 primary + 3 replica)	Redis memory > 12 GB lub połączeń > 10k
7. Dedykowane crawlery	~1 000+ klientów	Osobne VM dla kolejki `crawlers` — izolacja I/O sieciowego	Crawler jobs lag > 30 min

Narzędzia monitoringu i alertowania

CO MONITOROWAĆ	NARZĘDZIE	ALERT GDY
Kolejki Horizon	Laravel Horizon Dashboard	Lag > 5 minut, failed jobs > 10/godz.
Baza danych	pgBadger / pg_stat_statements	Slow queries > 1s, connections > 80%
Redis	Redis INFO / RedisInsight	Memory > 80%, evictions > 0
Serwery (CPU/RAM)	Netdata / Grafana + Prometheus	CPU > 80% przez > 5 min, RAM > 90%
WebSocket Reverb	Custom metrics → Grafana	Connections drop, reconnect storm
TimescaleDB chunks	timescaledb_information views	Chunk size > 10 GB, retencja niesprawna

💡 **Najważniejsza zasada przy skalowaniu**

Mierz zanim zainwestujesz. Każdy upgrade serwera powinien być poprzedzony tygodniem zbierania metryk. Najczęstszy błąd: upgrade app servera gdy prawdziwym wąskim gardłem jest nieoptymalne zapytanie SQL bez indeksu lub brak cache dla powtarzalnych wyników AI.